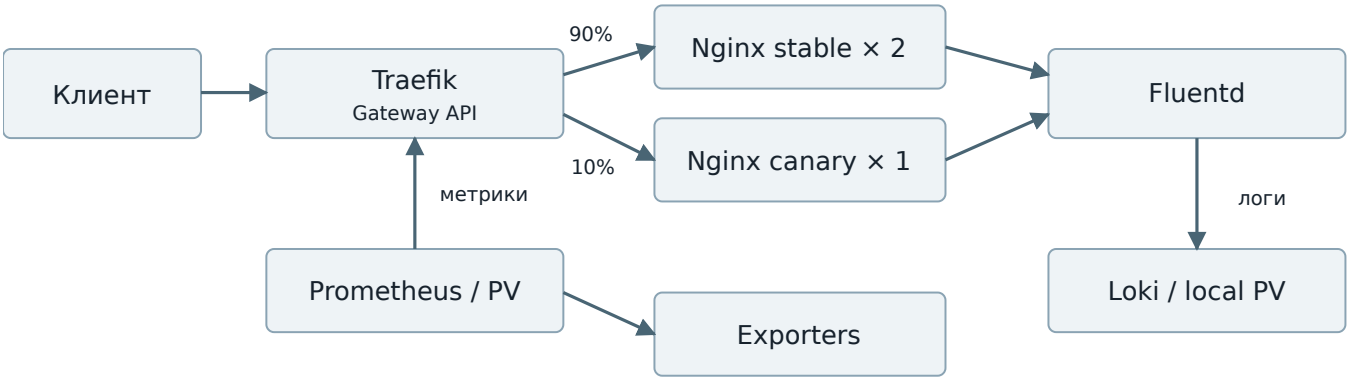
Self-Verifying Kubernetes Stack

Автоматическая установка и проверка веб-сервиса

Nginx возвращает `Hello World!` через Kubernetes Gateway API. Prometheus собирает метрики, Fluentd передает access/error-логи в Loki. Общий сценарий проверки связывает HTTP-ответ и запись лога по Request-ID и отдельно подтверждает прирост HTTP-счетчика.



На схеме показан canary-выпуск. После установки и после отката stable получает 100% трафика. Оба семейства IP доступны приложению; IPv6 — основной.

Слой	Состав
ОС и Kubernetes	Ubuntu 24.04.5 amd64; kubeadm / Kubernetes 1.34.3
Runtime и сеть	containerd 1.7.28; runc 1.3.3; Calico 3.31.2, dual-stack VXLAN
Gateway и приложение	Gateway API 1.4.0; Traefik 3.6.2 (chart 37.4.0); Nginx unprivileged 1.28.0-alpine
Метрики	kube-prometheus-stack 80.6.0; Prometheus 3.8.1; Operator 0.87.1; node exporter и kube state metrics
Логи	Fluentd 1.19.0 с Loki plugin; Loki 3.6.2
Автоматизация	Ansible core 2.19.3, Helm 3.19.0, Python и Make

> Проверенная конфигурация: одна VM, 4 vCPU, 8 GiB RAM, 60 GiB диска. Нужны SSH/sudo, IPv6/IPv4, DNS/NTP и доступ к публичным repositories/registries. Специфические облачные сервисы и инфраструктура автора не требуются.

Исходники и точные версии: README.md, config/versions.yaml, config/images.lock.yaml.

Функции и проверка

Перед запуском заполняется локальный environment и проверяется SSH. Далее `bash scripts/deploy.sh --env ubuntu` устанавливает инструменты, настраивает ОС и Kubernetes, затем разворачивает все компоненты.

Функция	Реализация и причина выбора	Как проверить
Kubernetes	kubeadm на Ubuntu: независимость от облачного вендора; Ansible настраивает runtime, сеть и данные.	`make health ENV=ubuntu`
Веб-приложение	Nginx с точным ответом и JSON access-логом. Простое приложение позволяет сосредоточиться на инфраструктуре.	`curl http://<VM_IPv4>:30080/`
Gateway API	Traefik, GatewayClass, Gateway и HTTPRoute к ClusterIP Service. NodePort не требует внешнего балансировщика.	`make verify-release ENV=ubuntu`
Prometheus	Operator и ServiceMonitor; targets Traefik, exporters и Prometheus. Проверяются свежие samples и прирост HTTP counter.	`make verify ENV=ubuntu`; API queries в инструкции
Fluentd / Loki	DaemonSet читает CRI stdout/stderr, передает в Loki. Positions и buffer сохраняются на узле.	Тот же verify находит access по Request-ID и отдельный error marker
Повторная установка	Закрепленные версии, SHA-256 и digest; проверка принадлежности кластера, сохранение PVC и Secrets.	`make verify-idempotency ENV=ubuntu`

Дополнительные возможности

Возможность	Реализация и проверка
IPv6 и IPv4	Обе семьи у Pod и собственных Services, IPv6 первый. Verify проверяет EndpointSlices, HTTP и логи обеих семей; один endpoint не собирается дважды.
HTTPS	Локальная CA и сертификат создаются автоматически, повторный deploy их сохраняет. Verify-release проверяет CA/hostname, включая отрицательные случаи.
Canary и откат	Два backend, 90/10, окно 60 с, минимум 100 canary-запросов, максимум 1% 5xx. Неверный ответ или недостаток метрик возвращают 100/0. Проверка: `make verify-canary ENV=ubuntu`.
Ограничения процесса	Nginx: UID 101, read-only root, без capabilities, privilege escalation и API token. Состояние проверяется в работающем контейнере.
Восстановление	Прерывание выпуска, замена app pod, история Prometheus/Loki и буфер Fluentd: `make demo ENV=ubuntu`. Reboot проверяется с внешнего controller.

Команды отказов предназначены для тестовой VM и временно меняют ее состояние. Отчеты сохраняются в artifacts; make evidence собирает их в архив. Полные команды HTTP, Prometheus и Loki — в docs/operations.md.

Условия успешной проверки и результаты: docs/verification.md.

Результаты и развитие

3 октября 2026 года выполнены две установки на чистую Ubuntu 24.04.5. Инструменты, Kubernetes и компоненты установил единый deploy; ручные исправления не потребовались.

Проверка	Результат
Чистая установка	5 мин 12 с и 5 мин 45 с
Локальные тесты и конфигурации	180 тестов; Python/Ansible/YAML, 7 Helm lint/render, image lock
Сквозной verify	HTTP, обе сети, метрики, access/error-логи, runtime security — pass
TLS/Gateway	154 проверки за прогон; неверные CA и hostname отклоняются
Повторный deploy	Сохранены 28 identities, привязки двух PVC и содержимое пяти Secrets
Canary и восстановление	Полный первый прогон: 6 canary-сценариев, interruption, app/PVC/buffer recovery — pass
Reboot	API восстановился за 34,85 с и 35,73 с; последующие health/verify — pass

Проверенные ревизии: c268c5f и 27d775b. Время подготовки VM и SSH не включено. Во втором прогоне сценарии canary и PVC/buffer recovery повторно не запускались. Машинные результаты и контрольные суммы сохранены в docs/evidence/.

Ревью работы

Главная особенность

Проверяется наблюдаемое поведение: правильный HTTP-ответ, изменение счетчика, точная запись в Loki и восстановление после отказа. Наличие YAML и Ready pod само по себе не считается достаточным результатом.

Самое сложное решение

При выпуске canary статус Kubernetes может обновиться раньше фактической конфигурации Traefik. Поэтому проверка ждет заголовок с примененными весами, проверяет readiness stable в трех точках и затем измеряет трафик без retries.

Как велась разработка

Автор задавал план и приоритеты; ИИ реализовывал конфигурации, скрипты и тесты. После разбора результатов уточнялись требования и исправлялись проверки адреса Gateway, image lock, выпуска canary и сохранения позиций Fluentd. Установка с чистой ОС была повторена отдельным поручением перед сдачей.

Следующие шаги

- NetworkPolicy с проверкой потоков, Grafana и Alertmanager с тестовым приемником уведомлений.
- CI с временным Kubernetes и теми же проверками; нужен runner с достаточными CPU/RAM.
- Несколько узлов, отказоустойчивое хранилище и backup/restore; для телеком-сценариев — проверка деградации и отказа каждой IP-семьи.

Ограничения: один узел, нет HA и backup. Потеря VM/диска означает потерю локальных данных; возможны дубли и потери логов при длительном сбое. TLS — локальная CA без автоматической ротации.

План и замечания: docs/implementation-plan.md и docs/development.md.